

FITBUDDY

AI-Powered Personalized Workout and Wellness Assistant

PHASE 02

Requirement Analysis Phase

Translate the FitBuddy concept into functional, non-functional, data, interface, and quality requirements.

Prepared for phase-wise submission
30 September 2026

Phase objective: Translate the FitBuddy concept into functional, non-functional, data, interface, and quality requirements.

1. Functional Requirements

- FR-01: Accept user profile data through the web form.
- FR-02: Validate user ID, username, age, weight, goal, and intensity before generation.
- FR-03: Generate exactly seven workout days.
- FR-04: Provide exercise, warm-up, cooldown, rest, and recovery information.
- FR-05: Generate nutrition, hydration, and recovery suggestions.
- FR-06: Save the user and plan in SQLite.
- FR-07: Accept feedback and produce a revised plan.
- FR-08: Display stored users in the local users view.
- FR-09: Allow local deletion of a user record.
- FR-10: Expose health and Swagger API endpoints.

2. Non-Functional Requirements

- Usability: the user flow should be understandable from a single home page.
- Maintainability: business logic is separated into routes, schemas, models, database, and AI service modules.
- Reliability: AI responses are validated against Pydantic schemas before being returned.
- Testability: the main flow runs in mock mode and is covered by automated tests.
- Security baseline: API keys belong in .env and the public-facing product should add authentication and authorization before deployment.

3. Input and Output Requirements

- Inputs: username, user ID, age, weight, goal, and intensity; feedback for revisions.
- Generated output: title, summary, safety note, seven workout days, and exercise details.
- Nutrition output: one nutrition tip, hydration guidance, and recovery guidance.
- Persistence output: User and Plan records containing original/revised plan JSON, feedback, and timestamps.

4. User Stories

- As a user, I want a 7-day plan based on my goal and intensity so that I can follow a weekly routine.
- As a user, I want nutrition and recovery guidance so that the plan covers more than exercises.
- As a user, I want to submit feedback so that the plan can be adjusted.
- As a project evaluator, I want a visible users page and API documentation so that the implementation can be demonstrated.

5. Constraints and Assumptions

- The default project database is SQLite for local development.
- The AI integration is configurable through environment variables.
- Mock AI mode is used for offline testing without a Gemini key.
- The generated content is general wellness guidance and should be reviewed before real-world use.

Requirement Traceability Snapshot

Requirement	Primary implementation area
User validation	app/schemas.py + app/routes.py
Workout generation	app/services/gemini_generator.py
Nutrition tips	app/services/gemini_flash_generator.py
Plan revision	app/services/updated_plan.py
AI client	app/services/gemini_client.py
Persistence	app/models.py + app/database.py
Web UI	templates/ + static/
Testing	tests/conftest.py + tests/test_app.py

Phase Deliverables / Evidence

- Functional and non-functional requirements
- Input/output definition
- Requirement traceability snapshot

Submission Note

This phase document is intended to be submitted together with the corresponding FitBuddy source project and evidence captured during execution. Keep API keys, personal credentials, and other secrets out of screenshots and submissions.